



LIVEQUEUE

We hold your spot. You own your time.

LiveQueue

A Real-Time Digital Queue Management Platform

Join from a QR code, wait anywhere, get called the moment it is your turn — with server-authoritative dispatch across many counters.

In production • Android v1.0.6

Presenter: [Your name] • Supervisor: [Name] • [Department, University] • [Date]

Queues are still physical and uncertain

1 Visitors

- Stand in a crowd for an unknown time
- No live update, easy to miss a call
- Multi-step visits restart the wait

2 Counter staff

- Who is next? Who was here first?
- Several counters, several services
- Hand-offs between counters are informal

3 Administrators

- No visibility of load or idle counters
- No record of who served whom
- Hard to run several desks or teams

Multi-service visits (e.g. an enquiry, then a payment) make every one of these worse.

One live queue, shared by every device

Organizations & roles

Head, Manager, Admin
workspaces, Executives

Queues & counters

Many counters per queue ·
Open / Paused / Off

QR entry

One organization QR →
choose a queue

Ordered journeys

Several services in a chosen
order · referrals

Live updates

Socket.io: position, ETA,
called, done

Notifications

Android FCM · iPhone Web
Push · email

Governance & audit

Admin replacement · Head
succession · immutable
history

Floating Counter Console

Always-visible controls (in
review)

Clients: Flutter Android app · React staff dashboard · iPhone/iPad Safari portal (PWA)

Four roles: governance and operations

GOVERNANCE

Organization Head

Whole organization · exactly one

Organization Manager

Org-wide reports & audit · no serving

OPERATIONS — isolated Admin workspaces

Admin A

Owens ≤ 1 live queue

Executive

Executive

Queue: Student Services

Admin B

Owens ≤ 1 live queue

Executive

Executive

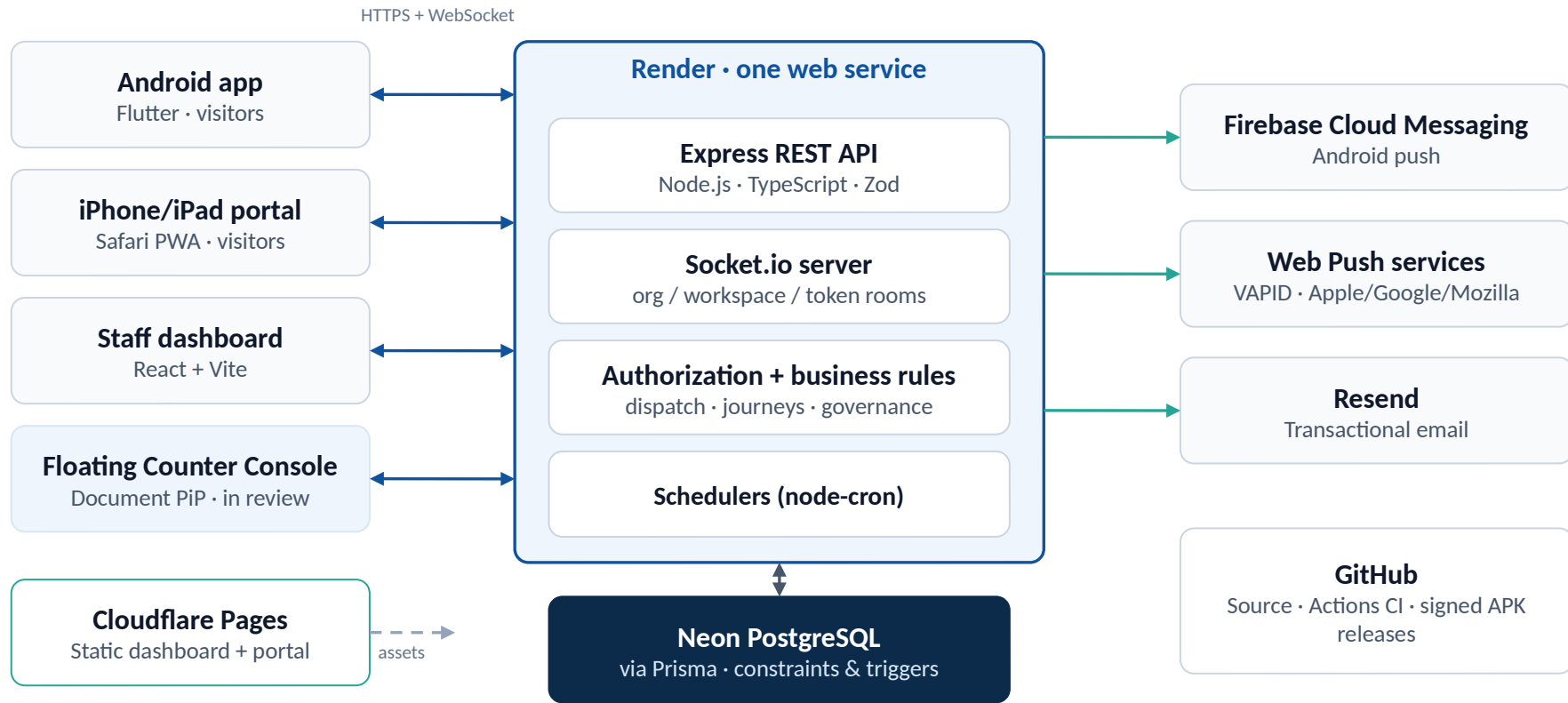
Queue: Library Desk

✗ no cross-workspace access

Product rules (enforced in the database)

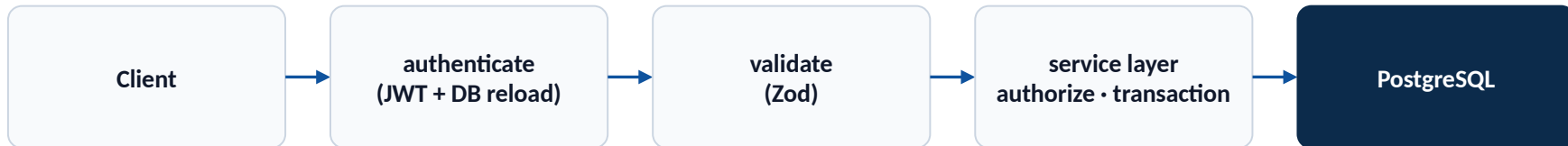
- One Head per organization
- One live queue per Admin
- A live queue always keeps its Admin
- Executives belong to one workspace
- Permissions derive from role only

System architecture

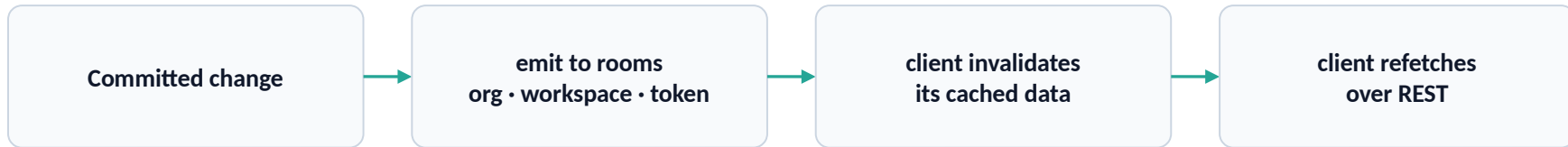


REST for truth, Socket.io for change

REST — every read and write

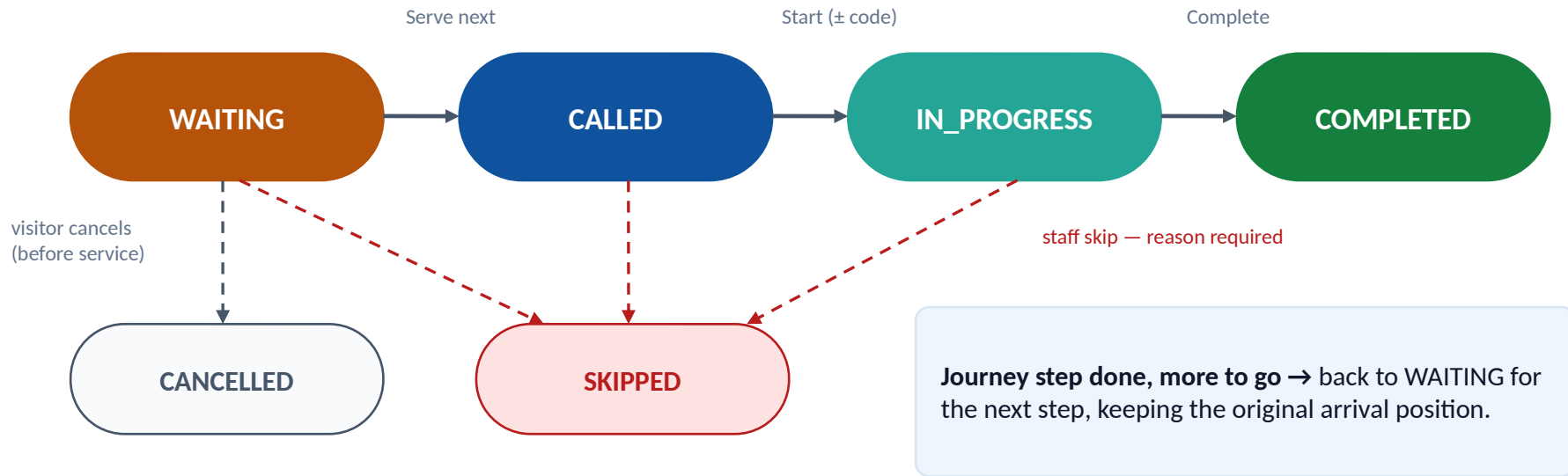


Socket.io — "something changed"



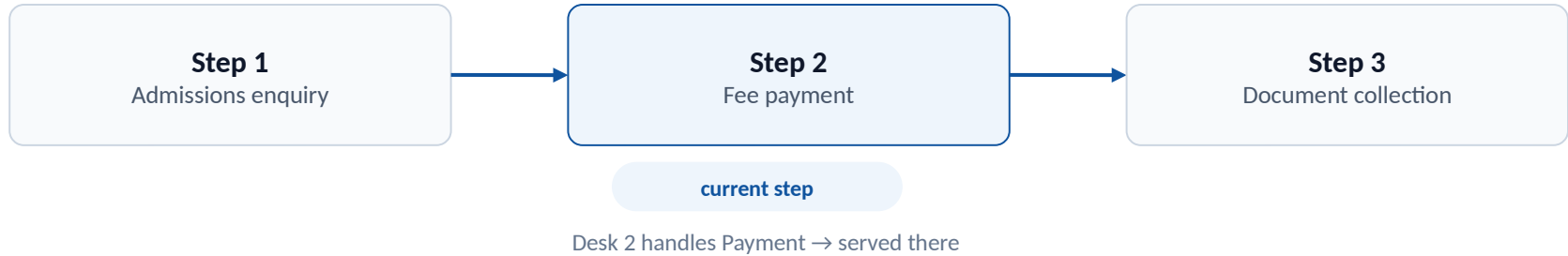
On reconnect: re-join rooms and refetch everything — a missed event is never assumed to be replayed. **Rooms:** organization (Head/Manager), workspace (Admin/Executives), token (one visitor), public queue (no personal data).

Token lifecycle: one server-side state machine



Serve next and Start are atomic compare-and-swap updates; the server rejects any transition the machine does not allow.

Ordered journeys and referrals



Ordered, 1-20 steps

Chosen by the visitor from the recommended order

Repeat rules

A service may repeat (limit 1-10), never twice in a row

Locked at creation

Any change is refused: 409 JOURNEY_LOCKED

Counter eligibility

A counter serves only the services it handles

Referral

Next step goes to a chosen open counter — no interruption

Fallback

Target closes → priority kept at another eligible counter

Many counters, one server-side dispatcher

Counter states

Open

ACTIVE · operator required

Paused

ON_BREAK · operator retained

Off

OFFLINE · unassigned

Assign operator to Off → Paused (never straight to Open). Off is refused while someone is called or in service there.

Serve next — decided on the server

1

Client sends only the queue id

No person, counter or staff id is accepted

2

Lock the caller's own counter

SELECT ... FOR UPDATE inside a transaction

3

Pick: referrals first, then earliest arrival

Whose current step this counter handles

4

Compare-and-swap WAITING → CALLED

A concurrent claim matches nothing and moves on

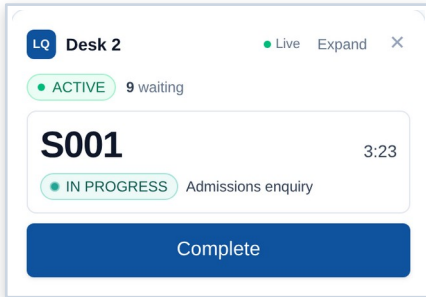
5

Commit, then broadcast

Every device refetches the same truth

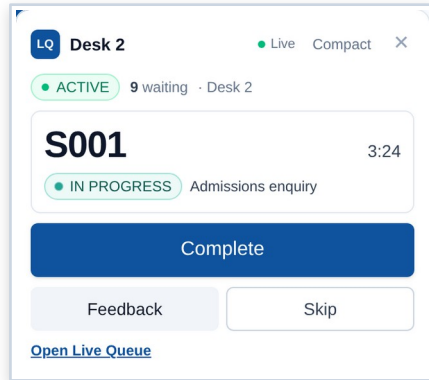
Floating Counter Console

Compact



Real screenshots, fictional demo data.

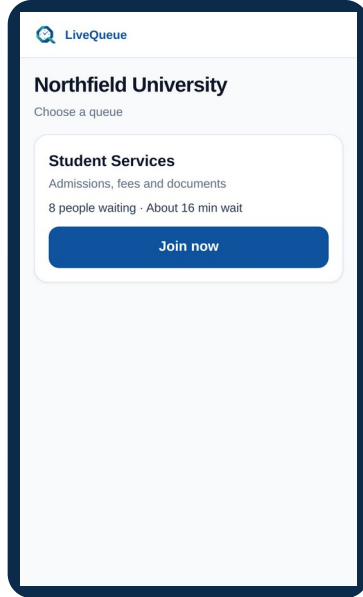
Expanded



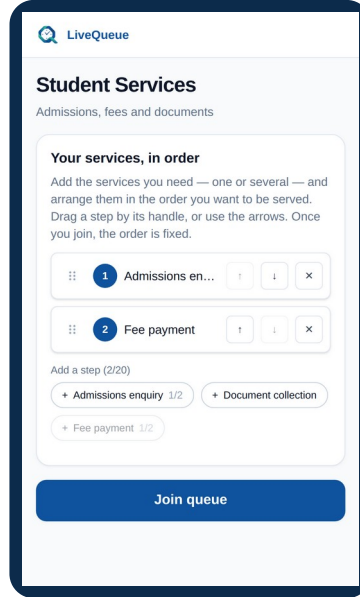
- Document Picture-in-Picture: a small always-on-top window (Chromium desktop browsers)
- Same APIs, same Socket.io connection — no second dispatch engine
- Shows token, service, state, waiting count, server-timed timer
- Actions held while reconnecting; access loss closes it
- Token number and service only — no personal data
- Fallback: in-page dock that says it cannot stay on top

Ready for review • not deployed

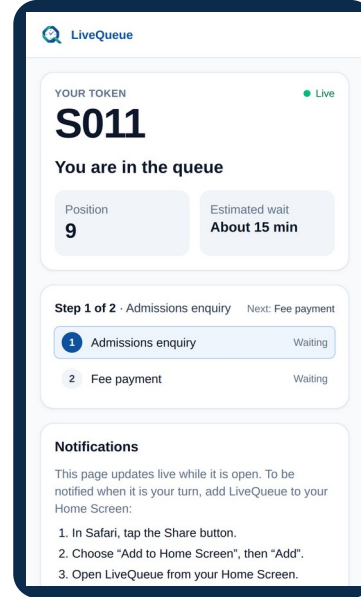
Visitor experience: QR → services → token



1 • Scan organization QR



2 • Choose services in order



3 • Live token, position, ETA

Android (Flutter)

Same flow in the app; push via Firebase Cloud Messaging.

iPhone / iPad

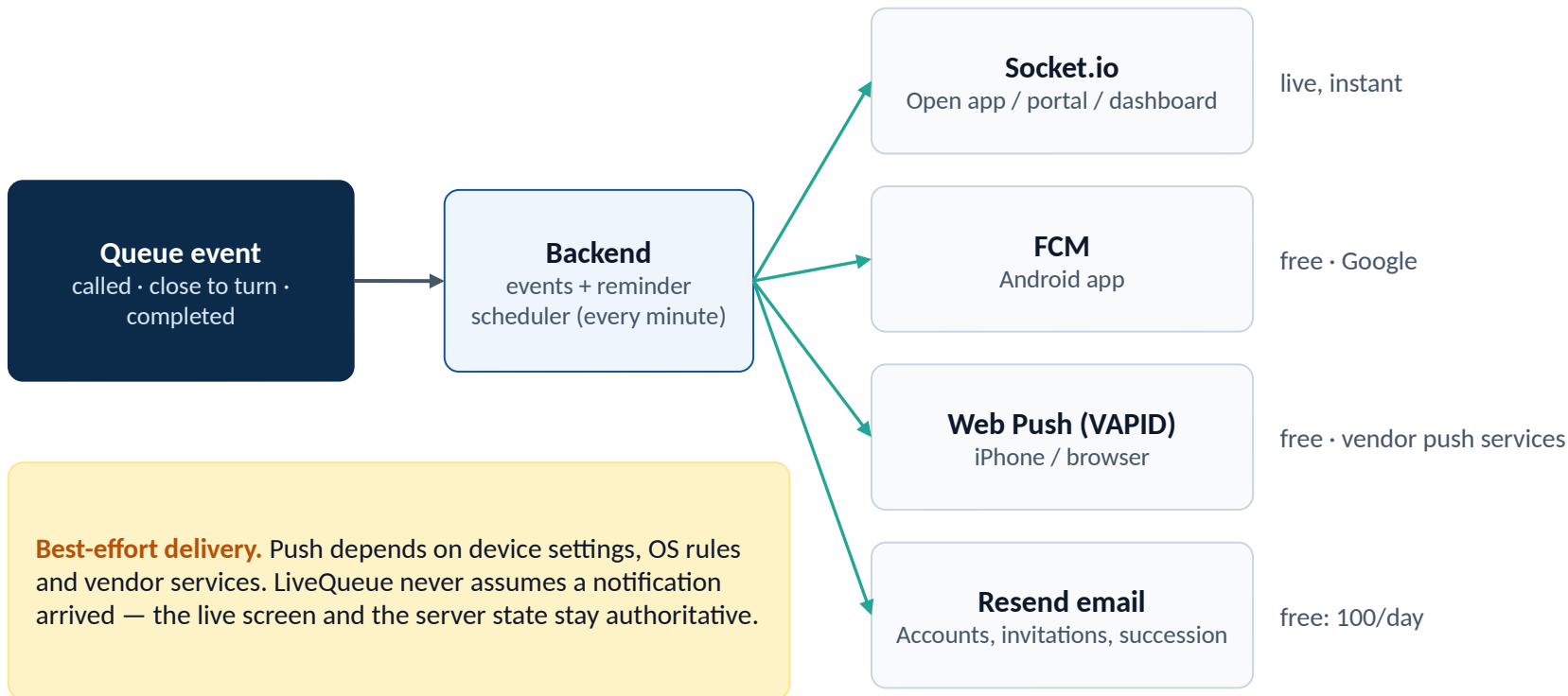
Safari portal; notifications need iOS 16.4+, Home Screen install and permission.

Physical iPhone push: pending

Physical Android QA: pending

Screens: fictional demo data.

Notification architecture



Governance and succession



Admin replacement

Queue + every Executive move atomically; counters, tokens and history untouched



Live queue keeps its Admin

Database trigger; refused with a guided replacement flow



Head succession

Password + emailed code → single-use 72-hour link → explicit acceptance



Tenure history

Who led the organization, when, and why — visible by role



Transactional audit

Change and audit row commit together, with actor snapshots



Immutable history

Triggers block edits and deletes; deleting an org leaves a minimal receipt

Security: implemented vs known gaps

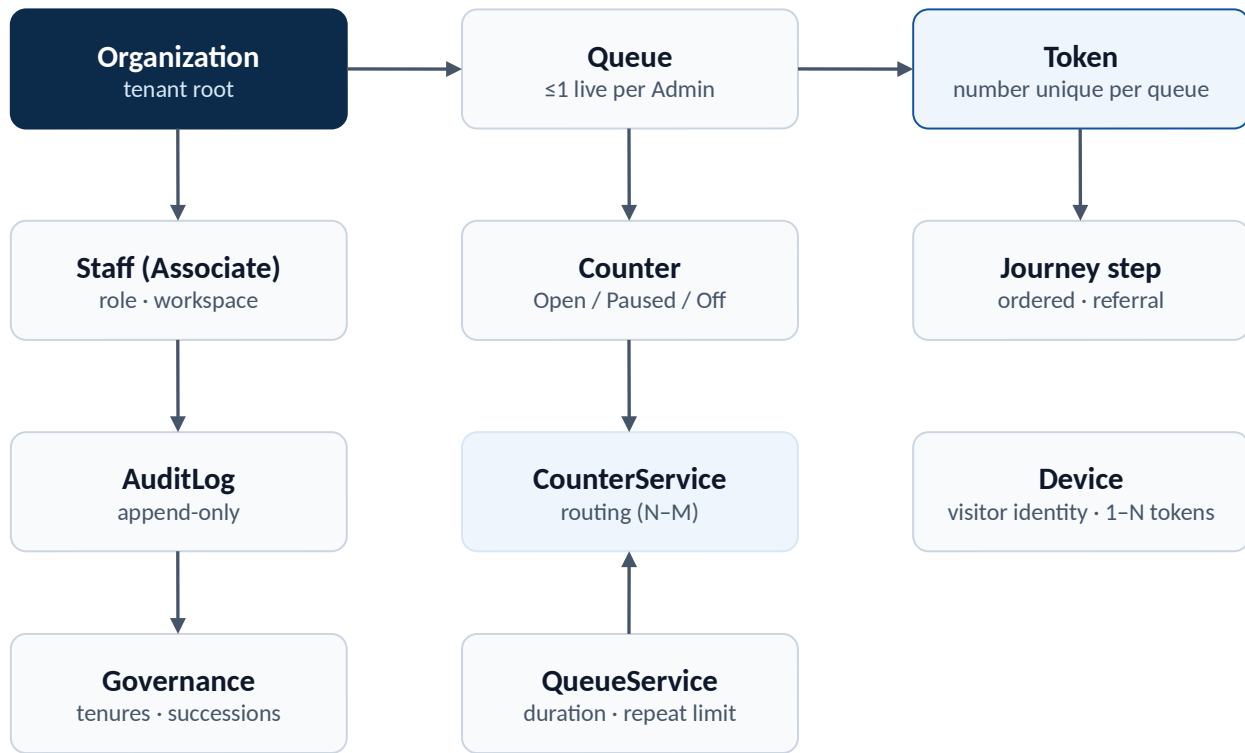
Implemented

- bcrypt passwords · 15-min JWT · rotated, hashed refresh tokens
- User + organization reloaded from the DB on every request
- Tenant & workspace scoping on every query (404, never leak)
- Backend authorization on every mutation, whatever the UI shows
- Instant revocation: tokens + live sockets dropped
- Single-use, hashed, expiring invitation and handover links
- Transactional, immutable audit with actor snapshots
- Rate limits on public, join, auth and sensitive routes

Known gaps

- Rate limits are in-memory, per process
- No Express "trust proxy" → limits may be shared across clients behind Render (to verify and fix)
- No external penetration test or WAF
- No centralized security monitoring or alerting

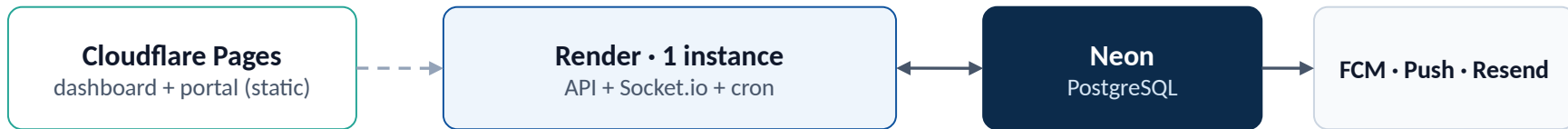
Database design (simplified)



Enforced in PostgreSQL

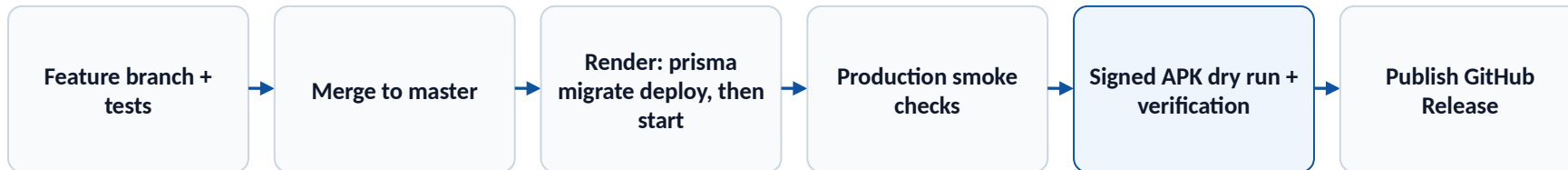
- One Head per org
- One live queue per Admin
- Live queue needs an Admin
- One open succession
- Unique token number per queue
- History append-only (triggers)

Deployment and release pipeline



Single instance by design: Socket.io has no cross-instance adapter yet.

Release process (used for v1.0.6)



APK verification on every release: exactly one signer • certificate pinned to 70:08:D3:....0A:93 • package com.livequeue.mobile_app • version 1.0.6 (build 7) • not debuggable • production API compiled in. Signing keys live only in GitHub secrets — never in the repository.

Testing and current production status

1,216

backend tests

574

dashboard tests

479

Flutter tests

All passing on production master; console branch: 1,222 + 614. Typecheck, lint and production builds clean.

Verified automatically

- Backend live: health, Socket.io handshake, Web Push config
- Both governance migrations applied; new routes respond
- Dashboard + portal render (headless Chromium checks)
- Android v1.0.6 published, Latest; APK independently verified

Pending / not yet done

- Real iPhone Web Push delivery — pending physical iPhone test
- Physical Android device QA — pending
- Formal load testing — not done
- Floating Console — in review, not deployed
- Post-deploy DB counts recorded manually by the owner (read-only SQL)

Capacity, cost and scaling

Answering “How big can it run for free?” — with every number labelled

What is actually free today?

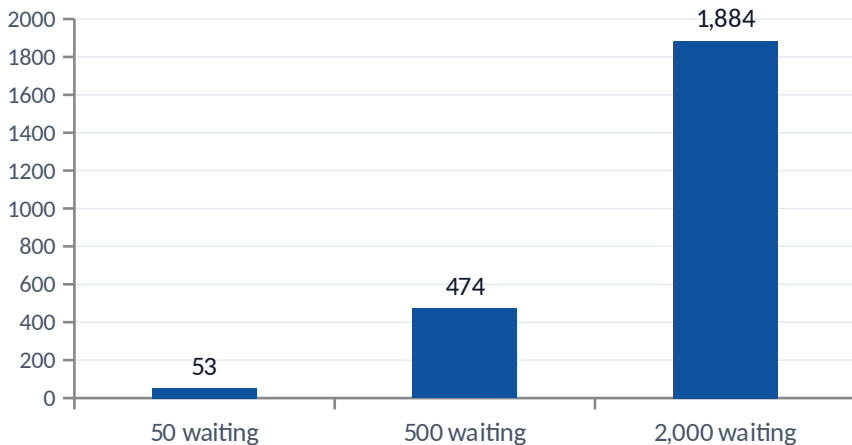
Service	Plan	Key free limits (PROVIDER LIMIT)
Render (API + Socket.io)	Free web · Hobby	0.1 CPU · 512 MB · 1 instance · sleeps after 15 min (≈1 min wake) · 5 GB outbound/month
Neon (PostgreSQL)	Free	100 CU-h/month (≈400 h at 0.25 CU) · 1 GB storage · 5 GB egress · sleeps after 5 min
Cloudflare Pages	Free	500 builds/month · no published static bandwidth cap
Firebase Cloud Messaging	No-cost	600k messages/min per project
Web Push (VAPID)	No fee	Vendor push services; device rules apply
Resend	Free	100 emails/day · 3,000/month
GitHub	Free (public repo)	Standard Actions runners free for public repos

Domain (tdastudbook.au): separate registration/renewal cost — not verified. Checked on: 2026-10-08

Registered ≠ daily active ≠ waiting now ≠ open sockets ≠ requests/second. Infrastructure is sized by concurrency and event rate, not by accounts.

Capacity: one queue

Socket bytes to one staff tab per serve cycle
(MEASURED, local)



ETA recompute per event (local): 16 ms · 103 ms · 454 ms

1 Serve next, 200 waiting → 305 dashboard requests (MEASURED)

	People waiting in one queue
Application limit	None (HARD LIMIT: 32-bit token number)
Comfortable today	≤ ~50 (ESTIMATE)
Caution	50 – 150 (ESTIMATE)
After refetch fix	~300 – 500 on free CPU (ESTIMATE)
Load-tested	NOT LOAD-TEST VALIDATED

Counters per queue: no limit; comfortable 1–10, caution 10–25 (ESTIMATE).

Capacity at every level

Level	Application rule (HARD LIMIT)	Comfortable (ESTIMATE)	Upgrade trigger
One counter	One operator · one person at a time	~12 visits/hour (5-min service)	—
One queue	No waiting limit	≤ ~50 waiting · 1–10 counters	> 150 waiting · > 25 counters
One organization	1 live queue per Admin; no Admin limit	1–2 queues · ~200 visits/day	≥ 5 busy queues
Whole platform	No organization limit	≈ 4,000 visits/month today · ≈ 25,000 after refetch fix	Egress or Neon hours exhausted

Assumptions: 4 queue events per visit, ~10 waiting, 2 staff tabs per queue, 8 h/day, 22 days/month. NOT LOAD-TEST VALIDATED.

Current bottlenecks, in order

0

Rate limits may be shared by all clients

Code finding: no Express "trust proxy" — verify; fix is free

1

Network egress: Neon 5 GB + Render 5 GB/month

Amplified by dashboard refetch per waiting person (software fix)

2

Neon compute: 100 CU-h $\approx$ 400 active hours/month

Reminder job keeps the DB awake while the backend is awake

3

Render Free: 0.1 CPU, ~1-minute cold start

Per-event work grows with queue length

4

Resend: 100 emails/day

Only if queues verify every visitor by email

—

Not limiting now: Cloudflare Pages, FCM, Web Push, GitHub Actions

Free or no published cap at this scale

ESTIMATE from measured payloads + published quotas. Checked on: 2026-10-08

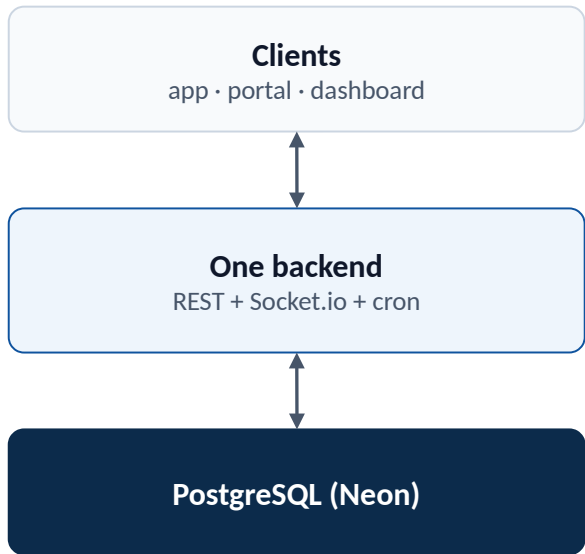
When do we need to pay?

When this happens	Service	Upgrade to	Why	Approx. cost
Refetch storm / 429s	Code	Debounce events; set trust proxy	Removes O(N) refetches	\$0
Neon egress or hours run out	Neon	Launch (usage-based)	500 GB egress, no hour cap	≈ \$11–20/mo*
Cold starts / CPU busy	Render	Starter 0.5 CPU	Always on, 5× CPU	\$7/mo†
Outbound > 5 GB	Render workspace	Payment method or Pro	Avoid suspension; 25 GB	\$0.15/GB or \$25/mo
Sustained load	Render	Standard 1 CPU · 2 GB	More compute, 1 instance	\$25/mo†
> 100 emails/day	Resend	Pro	50k/month, no daily cap	\$20/mo
Need > 1 instance	Render + Redis	Replicas + Socket.io adapter	Events reach every instance	Size after load test

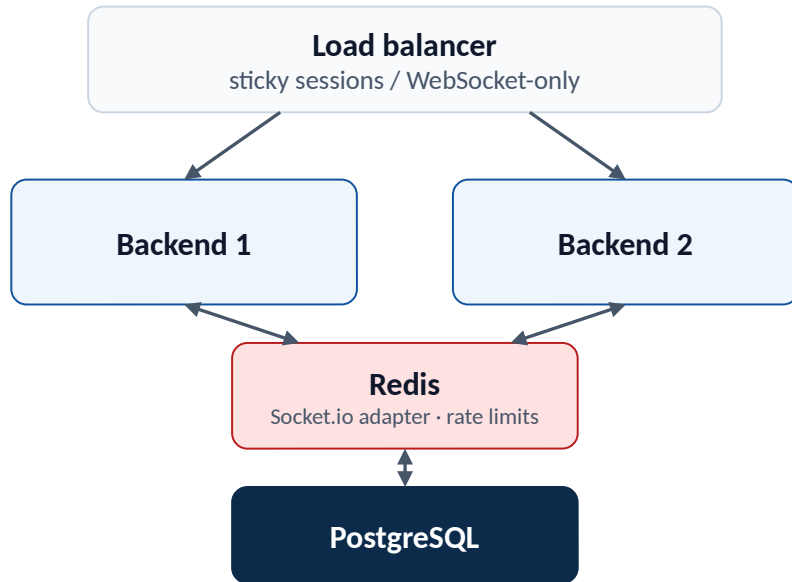
* 0.25 CU × 400–730 h × \$0.106/CU-h. † Render pricing page is dynamic; figures corroborated by third-party listings — confirm in dashboard. Checked on: 2026-10-08

From one backend to distributed realtime

TODAY



FUTURE (not implemented)



Adding instances alone breaks realtime: events stay on the instance that emitted them.

Cost & capacity roadmap

Stage 0

Today

\$0

All free tiers · 1 instance

≈ 4k visits/mo

Stage 1

Software first

\$0

Refetch fix · trust proxy

≈ 25k visits/mo

Stage 2

First paid

≈ \$18-27/mo

Neon Launch · Render
Starter

Always-on; no Neon hour cap

Stage 3

Growing multi-org

≈ \$70-110/mo

Render Standard · Pro
workspace · Resend Pro
(if needed)

More CPU + 25 GB
outbound

Stage 4

High concurrency

Size after load test

Replicas · Redis ·
Socket.io adapter ·
metrics

Horizontal realtime

ESTIMATES, not load-test validated. Fixed vs usage-based: Render plans fixed; Neon Launch usage-based. Domain cost separate. Checked on: 2026-10-08

Known limitations and future development

Limitations today

- Single backend instance; no Socket.io adapter
- Free-tier cold start (~1 minute after 15 idle minutes)
- Per-person event fan-out and dashboard refetch storm
- In-memory rate limits; trust proxy not set
- No formal load test; no central metrics/alerts
- Physical iPhone push and Android QA pending

Future (not implemented)

- Queue-level realtime updates instead of one per person
- Redis + Socket.io adapter + horizontal replicas
- Observability: metrics, central logs, alerts
- Analytics and predictive wait times
- Kiosk mode and institutional integrations
- Optional desktop (Tauri/Electron) counter companion

Live demo plan (≈ 5 minutes)

- 1 Log in as Head: dashboard, queue, counters, services
- 2 Visitor scans the organization QR on a phone
- 3 Build a 2-step journey; get a token
- 4 Executive presses Serve next — phone updates live
- 5 Start → Complete step → person returns for step 2
- 6 Referral to another counter (if time allows)
- 7 Open the Floating Counter Console (local build)
- 8 Show Associates, audit log and service history

Fallback: wake Render 5 minutes early · keep a local build ready · use the screenshots in this deck if a provider is down.

Conclusion

Real problem	Physical, uncertain queues → live, multi-counter, multi-service
Engineering	Server-authoritative dispatch · transactions · DB-enforced rules
Realtime	Socket.io across Android, iPhone and the dashboard
Trust	Instant revocation · immutable audit · governance & succession
Honest scale	Free today; first limit is egress (fixable); clear upgrade path

Questions?